



TEST RESULT AUTOMATION · PYTHON CLIENT

tcqrestclient

Programmer's Reference

PACKAGE

tcqrestclient 0.1.1

STATUS

Alpha · Second Edition

PUBLISHER

VIAVI Solutions

DATE

September 23, 2026

BACKEND

orion-res (TestCenter IQ)

© **2026 VIAVI Solutions Inc.** All rights reserved. This document describes `tcqrestclient`, an independently packaged, standalone Python client library for TestCenter IQ's `orion-res` results service. It has no runtime dependency on Spirent TestCenter® or Spirent HLTAPI®, which are referenced here only for identification and are the property of their respective owners.

The information in this document reflects `tcqrestclient` version **0.1.1 (Alpha)** as of **September 23, 2026**, and is subject to change without notice. Because several of the query builders described in Chapter 7 have been verified against real servers to differing degrees, every non-trivial claim in this manual carries one of four status markers, used consistently throughout:

CONFIRMED verified against a real `orion-res` server or GUI capture · **PARTIAL** implemented but only partly verified · **ASSUMED** reverse-engineered from frontend source, not yet captured live · **BROKEN** known to fail against a real server today

Treat any statement in this manual with no marker as ordinary, uncontested behavior.

Table of Contents

tcqrestclient Programmer's Reference · version 0.1.1

1	Introduction	
2	Installing and Configuring tcqrestclient	
2.1	Introduction	
2.2	Install Python	
2.3	Install tcqrestclient	
2.4	Configure the Connection	
2.5	Verify the Installation	
2.6	Connecting via the AION Platform (Optional)	
2.7	tcqrestclient Logs and Debug Output	
3	The IQClient	
3.1	Introduction	
3.2	Constructing a Client	
3.3	Configuration Resolution and Precedence	
3.4	The IQError Exception Family	
4	Discovering orion-res	
4.1	Introduction	
4.2	Local Discovery from an STC Install Directory	
4.3	AION Platform Discovery	
5	Managing Test Databases	
5.1	Introduction	
5.2	Listing and Inspecting Tests	
5.3	Database Schema Introspection	
5.4	Renaming and Deleting Databases	
5.5	Bulk Cleanup by Size or Age	

6	Named Views and Query Definitions	
6.1	Introduction	
6.2	Named Views vs. Ad-hoc Definitions	
6.3	View CRUD	
6.4	Discovering Columns and Resolving Fields	
7	Running Queries	
7.1	Introduction	
7.2	The query() Method	
7.3	Live vs. Snapshot (eot) Data	
7.4	Query Modifiers	
7.5	Supported View Types	
7.6	auto_repair and last_dropped_columns	
8	Generating Reports	
8.1	Introduction	
8.2	Report Templates	
8.3	Generating and Downloading a Report	
8.4	Downloading a Completed Report	
9	tcirestclient Python API Sample Usages	
9.1	Introduction	
9.2	Connecting and Discovering	
9.3	Managing Test Databases	
9.4	Views and Queries	
9.5	Reports	

10	Testing Automation Built on tcqrestclient	
10.1	Introduction	
10.2	Prerequisites	
10.3	Mocking orion-res with responses	
10.4	Fixtures and Environment Isolation	
11	IQClient Method Reference	
12	References	

Introduction

The **tcqrestclient Programmer's Reference** describes `tcqrestclient`, a standalone Python client for TestCenter IQ's `orion-res` results service. It covers installation and configuration, the client's object model, and every feature area of the API, including:

- Discovering an `orion-res` instance, either from a local Spirent TestCenter installation or through the AION platform
- Listing, inspecting, renaming, and cleaning up test databases
- Reading and cloning named views, and translating a view into an executable query
- Running table, live, snapshot, and chart queries with filters, sorting, grouping, and time ranges
- Generating and downloading reports from existing report templates
- Testing automation built on `tcqrestclient` without a live server

The target audience is automation engineers who write Python scripts to pull test results out of TestCenter IQ — for CI/CD result processing, ad-hoc analysis, or bulk database housekeeping. Readers are assumed to have:

- Working knowledge of Python 3
- Familiarity with REST APIs and JSON
- Some familiarity with TestCenter IQ's GUI concepts — tests, views, snapshots, and reports

No knowledge of Tcl, HLTAPI, or the legacy `stcrestclient` package is required.

`tcqrestclient` is packaged independently, in its own `tcqclient/` directory, with no dependency on either.

NOTE Every query in this API is a single synchronous HTTP request/response. There is no push, poll-loop, or WebSocket subscription mechanism anywhere in `tcqrestclient`. “Live” data simply means calling `query()` again to see a still-running test's current state — see 7.3, “Live vs. Snapshot (eot) Data.”

Installing and Configuring tciqrestclient

2.1 Introduction

This chapter covers everything needed to get `tciqrestclient` running: installing Python, installing the package itself, pointing it at an `orion-res` server, and confirming the installation. To install and configure `tciqrestclient`, complete the following steps:

1. Install Python 3.8 or later
2. Install `tciqrestclient`
3. Configure the connection (environment variables or a `.env` file)
4. Verify the installation
5. Connect via the AION platform (optional)

2.2 Install Python

`tciqrestclient` requires **Python 3.8 or later** (tested through 3.12 in continuous integration). Install Python from python.org or your platform's package manager, then confirm `pip` is available:

Windows / Linux / macOS

```
python --version
python -m pip --version
```

2.3 Install tciqrestclient

Install `tciqrestclient` with `pip`:

```
pip install tciqrestclient
```

From an internal package index instead of public PyPI:

```
pip install tciqrestclient --index-url <your-internal-index-url>
```

To install (or pin) a specific version:

```
pip install tciqrestclient==0.1.1
```

To upgrade an existing install to the latest available version:

```
pip install --upgrade tciqrestclient
```

NOTE The parent package also declares an `iq` extras group, so `pip install stcrestclient[iq]` installs both clients in one step from the same index.

2.4 Configure the Connection

Every `IQClient` constructor argument can also be supplied as an environment variable, or as an entry in a `.env` file loaded automatically at construction time. Nothing is hardcoded — a properly configured machine can call `IQClient()` with no arguments at all.

Copy `tciclient/.env.example` to `.env` and fill in whichever variables apply. Pick *one* of the four ways to locate `orion-res`:

ENVIRONMENT VARIABLE	CONSTRUCTOR ARGUMENT	PURPOSE
<code>TCIQ_BASE_URL</code>	<code>base_url</code>	Full base URL, e.g. <code>http://127.0.0.1:9200</code> . Skips discovery entirely; highest-priority address source.
<code>TCIQ_HOST</code> / <code>TCIQ_PORT</code>	<code>host</code> / <code>port</code>	Explicit <code>orion-res</code> address. <code>host</code> must always be paired with <code>port</code> — there is no dynamic port lookup for a bare host.
<code>TCIQ_INSTALL_DIR</code>	<code>install_dir</code>	Spirent TestCenter install directory; discovers the address from <code>orion-res.yaml</code> (local IQ) or <code>stcbll.ini</code> (remote IQ). See 4.2 .
<code>TCIQ_AION_URL</code> + <code>TCIQ_AION_USERNAME</code> + <code>TCIQ_AION_PASSWORD</code>	<code>aion_url</code> / <code>aion_username</code> / <code>aion_password</code>	AION platform discovery. Lowest-priority address source when resolved purely from environment variables. See 4.3 .

Optional variables, independent of which address source is used:

ENVIRONMENT VARIABLE	CONSTRUCTOR ARGUMENT	DEFAULT	PURPOSE
TCIQ_DATABASE_ID	database_id	unset	Default test (database) id for <code>query()/get_test()</code> when <code>use_test()</code> hasn't been called. Usually left unset in favor of calling <code>use_test()</code> at runtime.
TCIQ_TIMEOUT	timeout	10 seconds	HTTP request timeout; overridable per call via most methods' own <code>timeout=</code> .
TCIQ_DEBUG	debug	unset (off)	1/true/yes/on to log every request's method, URL, and JSON body before sending, and its status/timing after. See 2.7.
TCIQ_VERIFY_SSL	verify	unset (normal verification)	TLS verification for the <i>main</i> orion-res connection. <code>false/0/no/off</code> disables verification entirely; any other value is treated as a CA bundle file path.
TCIQ_AION_NODE_NAME	aion_node_name	unset (any node)	Restrict AION product-instance discovery to one node when several are present.
TCIQ_AION_PORT_NAME	aion_port_name	iq	Port-entry name orion-res appears under in AION's product-instances list.
TCIQ_AION_CA_CERT	aion_ca_cert	unset	CA bundle for the AION login itself — separate from <code>TCIQ_VERIFY_SSL</code> , which covers the orion-res connection.

SECURITY Only set `TCIQ_VERIFY_SSL` to disable verification for a deployment already trusted on your own network — a lab server with a self-signed certificate, for example. Never disable certificate verification for anything reachable over the open internet.

The full precedence order, from highest to lowest priority, is identical across every constructor argument:

1. Explicit `base_url` keyword argument
2. Explicit `host` / `port` keyword arguments
3. Explicit `install_dir` keyword argument

4. Explicit `aion_url` keyword argument (if AION discovery succeeds)
5. `TCIQ_BASE_URL` environment variable
6. `TCIQ_HOST` / `TCIQ_PORT` environment variables
7. `TCIQ_INSTALL_DIR` environment variable
8. AION discovery resolved purely from `TCIQ_AION_*` environment variables (lowest priority)

In short: **any explicit keyword argument always outranks the matching environment variable**. If nothing above resolves an address, construction raises `IQConfigError` naming every source it checked.

2.5 Verify the Installation

Confirm `tciqrestclient` installed and is importable, and check its version:

```
python -c "import tciqrestclient; print(tciqrestclient.__version__)"
```

To confirm end-to-end connectivity against a real (or configured) `orion-res` instance:

```
python
```

```
from tciqrestclient import IQClient

iq = IQClient()           # everything from .env / environment
print(iq.base_url)        # resolved orion-res base URL
print(iq.list_tests())    # raises IQRequestError if unreachable
```

Run any of the packaged examples the same way, from inside `tciqclient/`, after editing the connection settings and view/owner names at the top of the file to match your environment:

```
python examples/run_view_query.py
```

2.6 Connecting via the AION Platform (Optional)

When TestCenter IQ sits behind an AION platform deployment rather than being reached directly, supply AION credentials instead of a direct address. `tcqrestclient` authenticates to AION, discovers `orion-res`'s real host/port from AION's inventory, and attaches the resulting bearer token to every subsequent request automatically.

python

```
from tcqrestclient import IQClient

iq = IQClient(
    aion_url="https://aion.example.com",
    aion_username="user@example.com",
    aion_password="secret",
)
print(iq.base_url)  # the discovered orion-res address
```

See [Chapter 4, “Discovering orion-res,”](#) for the full AION discovery flow, including how to select a specific node when more than one AION-managed instance is present.

2.7 tcqrestclient Logs and Debug Output

There is no separate log file. Setting `debug=True` (or `TCIQ_DEBUG=1`) prints every request `tcqrestclient` makes directly to standard output, before it is sent, and its result immediately after:

console output, illustrative

```
-> POST http://127.0.0.1:9200/queries (timeout=10)
{
  "database": {"id": "12", "name": ""},
  "definition": {"multi_result": {...}}
}
<- POST http://127.0.0.1:9200/queries -> 200 in 0.184s
```

This is primarily useful for comparing a `tcqrestclient`-built request against the TestCenter IQ GUI's own DevTools network capture when a query behaves unexpectedly — see the note on filter qualification in [7.4](#).

The IQClient

3.1 Introduction

`IQClient` is the single public entry point of `tcqrestclient`. Every operation described in this manual — discovery, database management, views, queries, and reports — is a method on one `IQClient` instance:

```
from tcqrestclient import IQClient
iq = IQClient()
```

`IQClient` keeps track of your resolved connection settings, a default database id (set by `use_test()`), and a `last_dropped_columns` attribute populated by `query()`'s auto-repair mechanism (7.6).

3.2 Constructing a Client

tciqrestclient/client.py

```
class IQClient:
    def __init__(self, base_url=None, host=None, port=None,
                  install_dir=None, database_id=None, timeout=None,
                  use_https=False, load_env=True, env_file=None,
                  session=None, debug=None, aion_url=None,
                  aion_username=None, aion_password=None,
                  aion_node_name=None, aion_port_name=None,
                  aion_ca_cert=None, verify=None):
```

Every parameter falls back to its matching environment variable when omitted (2.4). The full, supported parameter list:

PARAMETER	DEFAULT	DESCRIPTION
<code>base_url</code>	None	Full base URL, e.g. <code>http://127.0.0.1:9200</code> . Skips discovery entirely; highest-priority address source.
<code>host</code>	None	Explicit orion-res host. Must always be paired with <code>port</code> .
<code>port</code>	None	Explicit orion-res port, paired with <code>host</code> .
<code>install_dir</code>	None	STC install directory used to discover the address (4.2).
<code>database_id</code>	None	Default test (database) id for <code>query()/get_test()</code> ; changeable later via <code>use_test()</code> .
<code>timeout</code>	10.0	HTTP request timeout, in seconds, applied to every request this client makes unless overridden per-call.
<code>use_https</code>	False	Compose a <code>host/port</code> or <code>install-dir</code> -discovered address with <code>https://</code> instead of <code>http://</code> . Ignored once a full <code>base_url</code> is given.
<code>load_env</code>	True	Load a <code>.env</code> file before reading environment variables.
<code>env_file</code>	None	Explicit <code>.env</code> path. None searches upward from the current directory (python-dotenv's default).
<code>session</code>	None	An existing <code>requests.Session</code> to reuse — mainly for tests (10.3).
<code>debug</code>	None	Log every request/response; falls back to <code>TCIQ_DEBUG</code> (2.7).
<code>aion_url</code>	None	AION platform base URL for AION-based discovery (4.3).
<code>aion_username</code>	None	AION login email/username.
<code>aion_password</code>	None	AION login password.
<code>aion_node_name</code>	None	Restrict AION product-instance search to one node when several are present.
<code>aion_port_name</code>	"iq"	Port-entry name orion-res appears under in AION's product-instances list.
<code>aion_ca_cert</code>	None	CA bundle path for the AION login itself — separate from <code>verify</code> below.
<code>verify</code>	None	TLS verification for the <i>main</i> orion-res connection: <code>True/False/a CA bundle path</code> . None (default) leaves a caller-supplied <code>session</code> 's own setting untouched rather than silently overriding it.

```
python
```

```
from tcqrestclient import IQClient

iq = IQClient(base_url="http://127.0.0.1:9200", timeout=20, debug=True)
print(iq.base_url)
```

Two module-level constants govern `query()`'s defaults and are re-exported from the top-level `tcqrestclient` package:

CONSTANT	VALUE	PURPOSE
DEFAULT_QUERY_LIMIT	1000	Default row limit for <code>query()</code> when <code>limit=</code> isn't passed — a safety net against pulling every row of a large test. Pass <code>limit=None</code> to leave a view's own limit untouched instead.
MAX_AUTO_REPAIR_ATTEMPTS	100	Cap on how many missing columns <code>auto_repair=True</code> will strip-and-retry before giving up. Confirmed against a real database missing roughly 21 columns at once.

3.3 Configuration Resolution and Precedence

When none of the address sources in 2.4 resolve to a usable `orion-res` address, constructing `IQClient()` raises `IQConfigError` before any network call is attempted. Its message enumerates every valid source you could supply instead: `base_url=`, `host=` / `port=`, `install_dir=`, or the `aion_url=` / `aion_username=` / `aion_password=` triplet — or their environment-variable equivalents.

```
python
```

```
from tcqrestclient import IQClient, IQConfigError

try:
    iq = IQClient(load_env=False)  # nothing to resolve an address from
except IQConfigError as e:
    print("Fix your configuration: %s" % e)
```

3.4 The IQError Exception Family

Every exception `tcqrestclient` raises subclasses `IQError`. All are importable directly from the top-level package. Catching the base class is usually enough for a script that just wants to report a failure and continue.

EXCEPTION	RAISED WHEN...
<code>IQConfigError</code>	No base URL could be resolved from any argument, environment variable, or <code>.env</code> entry.
<code>IQConnectionError</code>	Address discovery itself failed — a missing/unparseable <code>install-dir</code> config file, or any failure during AION login/lookup.
<code>IQRequestError</code>	An HTTP request to <code>orion-res</code> failed outright or returned a non-2xx status; also reused when <code>auto_repair</code> would strip every column from a query.
<code>IQQueryError</code>	A query call was malformed — both <code>name=</code> and <code>definition=</code> given (or neither), an invalid JSON <code>definition=</code> string, an unrecognized <code>test_live=</code> value, or an ambiguous/undeterminable <code>database_id</code> .
<code>IQViewError</code>	A named view could not be found, or its <code>view_type</code> is not one <code>query()</code> supports.
<code>IQReportError</code>	A report operation is missing a required argument — a <code>template_id</code> , <code>database_id</code> , <code>title</code> , or <code>report_id</code> .

python – catching the base class is usually enough

```
from tcqrestclient import IQClient, IQError

iq = IQClient()
try:
    rows = iq.query(name="Detailed Stream Results", limit=10)
except IQError as e:
    print("tcqrestclient call failed: %s" % e)
```

Discovering orion-res

4.1 Introduction

`tcqrestclient` supports two discovery mechanisms for locating an `orion-res` instance without being told its exact address: reading files inside a local Spirent TestCenter installation directory, and querying the AION platform's inventory API. Neither module reads environment variables directly — `tcqrestclient.config.resolve_config()` reads every `TCIQ_*` variable and passes the resolved values in as plain function arguments.

4.2 Local Discovery from an STC Install Directory

`tcqrestclient.discovery.discover_iq_address(install_dir)` makes no network call. It checks two files, in this fixed order, and returns the first usable address:

ORDER	FILE	KEY	USED FOR
1	<install_dir>/stcbl1.ini, section [enhancedResults]	orionResServicePublicUrl (preferred), else orionResServiceUrl	A remote IQ server — orion-res running on a different host than the STC install.
2	<install_dir>/orion- res/etc/orion-res.yaml	service.addr	A local IQ server — orion-res co-located with the STC install.

Either file may hold a bare `host:port` string or a full URL. `PyYAML` is imported only if `orion-res.yaml` is actually found. Both files use identical relative paths on Windows and Linux. If neither file yields an address, `discover_iq_address()` raises `IQConnectionError` naming both files and keys it checked, and suggesting `host=` / `port=` or `base_url=` as an explicit alternative.

```
python
```

```
iq = IQClient(install_dir=r"C:\Program Files\Spirent Communications\Spirent TestCenter")
print(iq.base_url)
```

4.3 AION Platform Discovery

Configuring `IQClient(aion_url=, aion_username=, aion_password=)` (2.6) runs a three-step flow against AION's REST API, and, unlike local discovery, comes back with a bearer token alongside the resolved address:

STEP	REQUEST	PURPOSE
1	GET /api/iam/organizations/default	Discover the organization id, used as the OAuth2 scope.
2	POST /api/iam/oauth2/token	OAuth2 password-grant login; captures access_token, refresh_token, and expires_in.
3	GET /api/inv/product-instances	List product instances; orion-res is identified as a port entry named TCIQ_AION_PORT_NAME (default iq) on each instance.

CONFIRMED The default AION port-entry name is `iq`, not `orion-res` — confirmed 2026-09-08 against a real AION organization with 58 product instances, 20+ carrying an `iq` port and zero named `orion-res`.

Pass `aion_node_name` to restrict the search to one AION node when several nodes or instances exist. If no matching port is found — on the matched node if one was given, or across every instance otherwise — discovery raises `IQConnectionError` naming the port it was looking for.

python – selecting a specific AION node

```
iq = IQClient(
    aion_url="https://aion.example.com",
    aion_username="user@example.com",
    aion_password="secret",
    aion_node_name="10.109.120.117",
)
```

Managing Test Databases

5.1 Introduction

Every `orion-res` “database” represents one test run’s results; **test** and **database** are used interchangeably throughout this manual, and `database_id` is the identifier used across the entire API. This chapter covers listing, schema inspection, and lifecycle management — implemented in `tcicrestclient/databases.py` and exposed on `IQClient`.

5.2 Listing and Inspecting Tests

METHOD	SIGNATURE	DESCRIPTION
<code>list_tests</code>	<code>list_tests(owner=None, detail="summary")</code>	Lists tests. <code>owner</code> filters <i>client-side</i> against <code>metadata["test.owner"]</code> — the server has no owner filter of its own.
<code>get_test</code>	<code>get_test(database_id=None)</code>	Full metadata for one test; uses <code>use_test()</code> ’s default when <code>database_id</code> is omitted.
<code>use_test</code>	<code>use_test(database_id)</code>	Sets the default database id for subsequent <code>query()/get_test()/generate_report()</code> calls.

`examples/list_tests_by_owner.py`

```
owner = "rakshit.bharti@viavisolutions.com"
tests = iq.list_tests(owner=owner)

for test in tests:
    print(" %s %-40s started=%s" % (
        test["id"], test["name"],
        test["metadata"].get("test.started", "?")))
```

5.3 Database Schema Introspection

METHOD	SIGNATURE	DESCRIPTION
<code>get_database_schema</code>	<code>get_database_schema(database_id=None)</code>	Full schema record: <code>id</code> , <code>datastore</code> , <code>name</code> , <code>description</code> , <code>metadata</code> , <code>dimension_sets</code> , <code>result_sets</code> , <code>profile</code> , <code>summary</code> .
<code>list_table_names /</code> <code>get_database_tables</code>	<code>list_table_names(database_id=None)</code>	Every table — result sets (fact tables) and dimension sets (lookup tables) — each tagged with a <code>kind</code> . <code>get_database_tables</code> is an identical alias.
<code>list_fields</code>	<code>list_fields(table_name=None, database_id=None)</code>	Field descriptors (<code>name</code> , <code>display_name</code> , <code>description</code> , <code>type</code> , <code>unit</code>), optionally scoped to one table.
<code>get_table_schema</code>	<code>get_table_schema(table_name, database_id=None)</code>	One table's own descriptor. Raises <code>IQError</code> if no table on this database has that name.
<code>get_database_summary</code>	<code>get_database_summary(database_id=None)</code>	Row count and storage size: <code>{"count", "value_storage_kb", "index_storage_kb"}</code> .

CONFIRMED A database's schema record has no top-level `tables` key. Table information lives under `result_sets` and `dimension_sets`; an early version of this client assumed a `tables` key and always returned an empty list until this was corrected.

examples/inspect_database_schema.py

```
tables = iq.list_table_names(database_id=test["id"])
fields = iq.list_fields(database_id=test["id"])
summary = iq.get_database_summary(database_id=test["id"])

print("%d table(s), %d row(s), %.1f KB total" % (
    len(tables), summary["count"],
    summary["value_storage_kb"] + summary["index_storage_kb"]))
```

5.4 Renaming and Deleting Databases

METHOD	SIGNATURE	DESCRIPTION
rename_test	rename_test(new_name, database_id=None)	Renames a database.
delete_test	delete_test(database_id=None)	Permanently deletes a database. Irreversible.

CONFIRMED `PUT /databases/<id>` does not accept a partial body. A body of just `{"name": new_name}` returns HTTP 400 (`RESOURCE_ID_MISMATCH`); adding a matching `id` field returns HTTP 500 with a Go nil-map panic. The only body that works is the *full* record: fetch the current record with a GET, change its `name` field, and PUT the whole thing back. `rename_test()` does this internally, so no caller-visible behavior changes — verified with a real rename-then-revert round trip.

WARNING `delete_test()` is destructive and irreversible, with no confirmation prompt built into the client. It exists primarily for CI/CD pipelines to clean up results after processing them.

examples/manage_test_database.py

```
before = iq.get_test(database_id=TARGET_DATABASE_ID)
updated = iq.rename_test(before["name"] + " (renamed)", database_id=TARGET_DATABASE_ID)
print("Renamed to: %s" % updated["name"])

# Irreversible -- confirm with the user before ever calling this for real:
iq.delete_test(database_id=TARGET_DATABASE_ID)
```

5.5 Bulk Cleanup by Size or Age

Four methods pair a non-destructive preview with a bulk delete, and both delete variants default to `dry_run=True`:

PREVIEW	BULK DELETE	CRITERION
<code>list_databases_over_size(min_size_kb)</code>	<code>delete_databases_over_size(min_size_kb, dry_run=True)</code>	Total storage (value_stored + index_stored) above a KB threshold.
<code>list_databases_older_than(days)</code>	<code>delete_databases_older_than(days, dry_run=True)</code>	metadata["created"] more than days past.

examples/bulk_cleanup_databases.py

```
matches = iq.list_databases_over_size(MIN_SIZE_KB)
if _confirm(matches, "at or over %d KB total storage" % MIN_SIZE_KB):
    iq.delete_databases_over_size(MIN_SIZE_KB, dry_run=False)
    print("Deleted.")
```

Both preview methods return the same shape the bulk-delete call would have deleted, whether or not deletion actually ran — always inspect the preview list before passing

`dry_run=False`.

Named Views and Query Definitions

6.1 Introduction

A **view** is a saved GUI query, identified by id or name. A **query definition** is the actual executable tree POSTed to `/queries`. This chapter covers both, and the translation step that bridges them — implemented across `tcqrestclient/views.py` and `tcqrestclient/view_query_builder.py`.

6.2 Named Views vs. Ad-hoc Definitions

A view's `details` field is a **GUI table specification** — a `view_type`, plus one or more `tables[]`, each naming a `query_provider` by id along with its active columns. It is a description of what the GUI shows, and it is *not* itself something that can be POSTed to `/queries`.

CONFIRMED Confirmed from a real server export of 449 views: a view's `details` is a GUI table spec, not a `multi_result` query definition. An earlier version of this client assumed the latter and was wrong.

An ad-hoc **query definition** (`tcqrestclient/query.py`, `tcqrestclient/queries.py`) is a tree keyed by one of `multi_result`, `single_result`, `single_dimension`, `raw`, or `delete_result`, containing nested `subqueries`, `projections`, `filters`, `groups`, `orders`, `limit`, and `pagination`. Projections, filters, and orders are literal, SQL-ish expression strings — e.g. `"view.frame_count>1000"` — not a structured `{field, op, value}` shape. That is simply how the real API works.

`tcqrestclient` bridges the two automatically: when you call `iq.query(name=<a view>)` (7.2), it walks the view's active columns through its own query templates to produce the same executable tree the GUI's own request would send — you never build that tree by hand. A hand-built or captured definition can skip views entirely and go straight to `iq.query(definition=...)`.

CONFIRMED The table-building translation is confirmed byte-for-byte against two real captured queries — one unfiltered, one with a filter added — not guessed.

6.3 View CRUD

METHOD	SIGNATURE	DESCRIPTION
<code>list_views</code>	<code>list_views(timeout=None)</code>	All views. Each includes its full <code>effective_details.system_data.query_providers</code> — on a server with many views this can be large and slow; override <code>timeout=</code> for just this call.
<code>get_view</code>	<code>get_view(view_id, timeout=None)</code>	One view by id.
<code>find_view</code>	<code>find_view(name, timeout=None)</code>	One view by display name (client-side scan of <code>list_views()</code>), or None.
<code>save_view</code>	<code>save_view(name, details=None, description="", definition=None)</code>	Create or update a view. <code>definition</code> is accepted as an alias for <code>details</code> .
<code>delete_view</code>	<code>delete_view(view_id=None, name=None, timeout=None)</code>	Delete a view. Exactly one of <code>view_id</code> or <code>name</code> must be given.

examples/manage_views.py — clone, query, then clean up

```
created = iq.save_view(
    name=EXAMPLE_VIEW_NAME,
    details=source["details"],
    description="Created by tciqurestclient's manage_views.py example -- safe to
delete.",
)
try:
    rows = iq.query(name=EXAMPLE_VIEW_NAME, data_type="eot", limit=5)
finally:
    iq.delete_view(view_id=created["id"])
```

CONFIRMED A view created via `save_view()` never gets its own `query_providers` populated by the server, even though its tables still reference one by name. `tciqurestclient` works around this by scanning every other view on the server for a matching provider, preferring one whose default sort order is non-empty — a naive first match was confirmed to sometimes return a provider with no default order, which then made a paginated query fail with `"pagination requires at least one order expression."`

`save_view()`'s full parameter list:

PARAMETER	DEFAULT	DESCRIPTION
name	required	The view's display name.
details	None	The view's raw GUI table spec, exactly as details appears on an existing view — typically read via <code>get_view()/find_view()</code> , modified, then passed back rather than built from scratch.
description	" "	Optional free-text description.
definition	None	Alias for details; takes precedence if both are given.

6.4 Discovering Columns and Resolving Fields

`list_view_columns()` lists every column-like attribute a view's table exposes — raw name, alias, and GUI `display_name` — to see what `query()`'s `filters=` / `sort=` / `group_by=` will accept. Its full parameter list:

PARAMETER	DEFAULT	DESCRIPTION
<code>name</code>	<code>required</code>	View name, resolved the same way as <code>query(name=)</code> .
<code>test_live</code>	<code>None</code>	Which table (live/eot) to list columns for — same normalization as <code>query()</code> 's <code>test_live=</code> .
<code>active_only</code>	<code>False</code>	<code>False</code> : every column the view's <code>query_provider</code> supports. <code>True</code> : only columns actually active on this table.
<code>timeout</code>	<code>None</code>	Per-call HTTP timeout override, applied to the <code>/views</code> lookup.
<code>data_type</code>	<code>None</code>	Advanced: select the table by exact <code>data_type</code> string instead of <code>test_live=</code> .
<code>table_index</code>	<code>None</code>	Advanced: select the table by raw position, bypassing <code>test_live=</code> / <code>data_type=</code> .

python

```
columns = iq.list_view_columns("Detailed Stream Results", active_only=True)
for col in columns[:3]:
    print(col["display_name"], "->", col["alias_name"])
```

Internally, `view_query_builder.build_field_resolver()` maps a column's GUI display name (case-insensitive), raw attribute path, unambiguous bare suffix, or already-correct alias to the alias `merge_modifiers()` needs. A bare name such as `frame_count` only resolves when exactly one active column ends in it — a view with both `tx_stream_stats.frame_count` and `rx_stream_stats.frame_count` active leaves the bare name unresolved rather than guessing.

Running Queries

7.1 Introduction

`IQClient.query()` is the **only** query method in `tcicrestclient`. Rather than exposing a different call per GUI widget type, it inspects a view's own `view_type` internally and dispatches accordingly.

7.2 The query() Method

tciqrestclient/client.py

```
def query(self, name=None, test_live=None, database_id=None,
          user=None, snapshot_name=None, filters=None, sort=None,
          group_by=None, time_range=None, limit=DEFAULT_QUERY_LIMIT,
          mode="once", raw_result=False, timeout=None, auto_repair=True,
          definition=None, data_type=None, table_index=None):
```

Exactly one of `name=` or `definition=` must be given. The full, supported parameter list:

PARAMETER	DEFAULT	MEANING
name	None	Name of an existing view to build a definition from. Exactly one of name/definition is required.
definition	None	Escape hatch: run a raw query definition (dict or JSON string) directly, bypassing views entirely. Exactly one of name/definition is required.
test_live	None	True for live data; False or omitted (default) for snapshot/eot. Also accepts the case-insensitive strings "live" / "eot" / "snapshot". See 7.3 .
database_id	None	Which test's results to query. Defaults to use_test()'s value or TCIQ_DATABASE_ID.
user	None	Auto-finds database_id instead of passing it — but only for test_live=True, by finding the one running test the user owns. Raises IQQueryError for snapshot data (a user can have many completed tests, with nothing to disambiguate).
snapshot_name	None	With name= only: restricts to one named snapshot of a finished (eot) test. Raises IQViewError against a live table.
filters	None	Row filters, layered on by merge_modifiers(). See 7.4 .
sort	None	Row sort order, layered on by merge_modifiers(). See 7.4 .
group_by	None	Grouping field(s), layered on by merge_modifiers(). See 7.4 .
time_range	None	Time-bounded filter, layered on by merge_modifiers(). See 7.4 .
limit	DEFAULT_QUERY_LIMIT (1000)	Row cap. limit=None leaves the view's/definition's own limit untouched instead of overriding it.
mode	"once"	Query execution mode. "once" is the only mode that exists — every query is one synchronous request/response (7.3).
raw_result	False	False returns row dicts; True returns the raw result object — id, columns, rows, pagination, timing.
timeout	None	Per-call HTTP timeout override, in seconds, applied to the /queries request(s) and, with name=, the /views lookup.
auto_repair	True	With name= only. True silently strips and retries past “unknown attribute” errors. See 7.6 .
data_type	None	Advanced: select the view's table by exact data_type string; takes precedence over test_live= if both are given.

PARAMETER	DEFAULT	MEANING
table_index	None	Advanced: select the table by raw position, bypassing test_live=/data_type=.

python – the minimal call

```
rows = iq.query("Detailed Stream Results", False, limit=100)
for row in rows[:3]:
    print(row)
```

Return shape depends on the view’s `view_type`. A single-query type (`table`, `x_y_chart`, `pie_chart`, or any `definition=` call) returns a plain list of row dicts, or the raw result object if `raw_result=True`. A “multi” type — `histogram` (one query per provider) or `boxplot` (one query per statistic) — returns a `dict` of `{<name>: <rows>}` instead.

7.3 Live vs. Snapshot (eot) Data

BY DESIGN Every query — live or snapshot — is one request/response: `mode="once"` returns the result synchronously in the same call. There is no WAMP, polling, or subscription mechanism anywhere in this API. “Live” only means the caller re-runs `query()` when fresh data is wanted.

Concretely, live vs. snapshot (also called **eot**, end-of-test) selects which *table* of a view is targeted, distinguished by that table’s own `data_type`: a `live` table serves a still-running test; an `eot` table serves the finished test’s stored results. When neither `table_index=` nor `data_type=` is given, `tcicrestclient` defaults to the table with `data_type=="eot"`, not whichever table a view happens to list first.

CONFIRMED Defaulting to `eot` rather than the first-listed table was a real bug fix (2026-09-03): for the “Detailed Stream Results” view, the first table listed was in fact its `live` table, so an unqualified call was silently returning live data instead of the documented default of snapshot data.

examples/run_live_query.py – polling a running test

```
for i in range(POLL_COUNT):
    rows = iq.query(VIEW_NAME, "live", user=OWNER, limit=10, timeout=30)
    print("%d row(s)" % len(rows))
    time.sleep(POLL_INTERVAL_SEC)
```

7.4 Query Modifiers

`tcicrestclient.query.merge_modifiers(definition, filters, sort, group_by, time_range, limit, resolve_field)` deep-copies a definition and layers caller-supplied modifiers onto its outermost node — the same mechanism whether the definition came from a view or was hand-built. It never mutates the original.

MODIFIER	ACCEPTS
<code>filters</code>	A raw expression string, a <code>(field, value)</code> or <code>(field, op, value)</code> tuple, or a <code>{"field", "op", "value"}</code> dict. <code>op</code> defaults to <code>eq</code> ; one of <code>eq/ne/lt/lte/gt/gte/contains</code> (LIKE '%value%')/in.
<code>sort</code>	A field name, <code>"field ASC"/"field DESC"</code> , or <code>(field, order)</code> — single or as a list.
<code>group_by</code>	A field name or list of field names.
<code>time_range</code>	A <code>(field, start, end)</code> tuple or <code>{"field", "start", "end"}</code> dict. Either bound may be omitted for an open-ended range; <code>field</code> is always required — there is no universal timestamp column across views.
<code>limit</code>	Replaces (not appends to) the existing limit; adds a default forward-pagination stanza the first time a limit is set.

With `name=`, a field may be given as its GUI display name (e.g. `"Rx Count"`, case-insensitive), its raw attribute path, an unambiguous bare column name, or its internal alias — all resolve to the same column via `list_view_columns()` / `build_field_resolver()`.

examples/query_modifiers.py

```
rows = iq.query(
    name=VIEW_NAME, data_type="eot",
    filters=[("Rx Count", "gt", 0)],
    sort="Rx Count DESC",
    limit=10, timeout=60)
```

Once a result is fetched, `tcicrestclient.query.rows_to_dicts(result)` zips the raw result's positional `columns` and `rows` into per-row dicts — tolerating a missing, empty, or `None` result by returning an empty list rather than raising. This is the shaping step `query()` already applies for you unless `raw_result=True`.

NOTE Field qualification against a view's outermost alias is not guaranteed to be byte-identical to how the GUI's own filter box places the same filter internally. If a filtered `name=` query doesn't behave as expected, capture the GUI's own request with `debug=True` (2.7) and compare.

7.5 Supported View Types

`query(name=...)` dispatches on a view's `view_type`. Table types are confirmed byte-for-byte against real captures; the four chart types were ported from `magellan-frontend`'s own TypeScript widget models rather than captured, and carry differing confirmation status. Any `view_type` not listed here raises `IQViewError`.

VIEW_TYPE	RETURN SHAPE	STATUS	NOTES
<code>single_level_table</code>	list of row dicts	CONFIRMED	Byte-for-byte against two real captures.
<code>paged_single_level_table</code>	list of row dicts	CONFIRMED	Same builder as <code>single_level_table</code> .
<code>x_y_chart</code>	list of row dicts	BROKEN	Real shape uses a <code>series[]</code> key with no <code>tables</code> key at all — confirmed to raise <code>IQViewError</code> on 3 independent real servers.
<code>pie_chart</code>	list of row dicts	ASSUMED	Mandatory group/total columns assumed from frontend source; not yet checked against a real provider export.
<code>histogram</code>	dict of <code>{provider: rows}</code>	BROKEN	Real shape uses <code>statistics[]/group_by</code> , no <code>tables</code> key — confirmed broken like <code>x_y_chart</code> .
<code>boxplot</code>	dict of <code>{statistic: rows}</code>	ASSUMED	Per-statistic split is not yet derivable from a view's own <code>effective_details</code> in any confirmed way; returns the right shape as a placeholder.
everything else (<code>chart</code> , <code>gauge</code> , <code>health_indicator</code> , <code>event_dashboard</code> , ...)	—	NOT SUPPORTED	Not backed by a row-returning query in a confirmed way, or not checked at all.

A raw `definition=` call bypasses this dispatch entirely and always returns a plain list of row dicts — it is the reliable escape hatch when a view's chart type isn't (yet) supported.

```
python - checking view_type before deciding how to consume the result
```

```
view = iq.find_view("Latency Distribution")
if view["details"]["view_type"] == "histogram":
    rows_by_provider = iq.query(name=view["name"], data_type="eot", limit=100)
    # {'provider_name': [rows...], ...} -- not a plain list
```

7.6 auto_repair and last_dropped_columns

A view's `query_provider` template is shared across every database that uses that view — but not every database's schema has every attribute the template can reference. A single-stack test's database, for example, genuinely lacks the dual-stack columns a shared template still lists. `orion-res` reports this with an HTTP 400 `VALIDATION_FAILED` response, one column at a time:

```
real orion-res error body
```

```
{"code": "VALIDATION_FAILED", "message": "Validation failed: name error; unknown attribute name: tx_stream_config.ipv4_2_source_addr as tx_stream_config_ipv4_2_source_addr"}
```

With `auto_repair=True` (the default, and only meaningful with `name=`), `query()` parses this message, strips the offending column from the query at every level it appears, and retries — up to `MAX_AUTO_REPAIR_ATTEMPTS` (100) times. If stripping would leave the outermost query with zero projections, it raises `IQRequestError` naming every column it had already dropped, rather than sending an empty query.

```
examples/auto_repair_demo.py
```

```
rows = iq.query(name=VIEW_NAME, data_type="eot", limit=50, timeout=60)
if iq.last_dropped_columns:
    print("Columns silently dropped to make this query succeed:")
    for col in iq.last_dropped_columns:
        print("  %s" % col)
```

`iq.last_dropped_columns` reflects whatever was actually stripped on the most recent call — a plain list for single-query view types, or a `dict` keyed the same way the returned rows are for `histogram` / `boxplot`. Nothing is printed automatically; check this attribute to know whether a query's result is missing columns it would otherwise have had.

Generating Reports

8.1 Introduction

This chapter covers generating a report from an existing report template and downloading its output file — implemented in `tcicrestclient/reports.py`.

8.2 Report Templates

METHOD	SIGNATURE	DESCRIPTION
<code>list_report_templates</code>	<code>list_report_templates()</code>	All available
<code>get_report_template</code>	<code>get_report_template(template_id)</code>	One template
<code>find_unsupported_report_sections</code>	<code>find_unsupported_report_sections(template_id, timeout=None)</code>	Which of a whose view report rend generate_

CONFIRMED Including a chart-type (`x_y_chart`) section in a generated report crashes the report-rendering frontend with a reproducible `TypeError` — root-caused directly against real `orion-res / magellan-frontent` source. `single_level_table`, `paged_single_level_table`, and `drill_down_table` sections are confirmed safe; every other view type is treated as unsafe until checked.

python

```
templates = iq.list_report_templates()
template = templates[0]

unsupported = iq.find_unsupported_report_sections(template["id"])
print("Sections to exclude:", unsupported)
```

8.3 Generating and Downloading a Report

tcicrestclient/client.py

```
def generate_report(self, template_id, title=None, database_id=None,
                    database_name=None, format="pdf", snapshot_filter=None,
                    excluded_sections=None, output_path=None, **extra):
```

PARAMETER	DEFAULT	NOTES
template_id	required	Id of an existing report template.
title	None	Required in practice — a real server 400s with “missing report title” if omitted. tcicrestclient raises IQReportError before ever sending the request.
database_id	None	Defaults to use_test()'s value or TCIQ_DATABASE_ID via _require_database_id().
database_name	None	Optional; when given, included as database.name alongside database.id (matches the real GUI request shape). Omitted from the body entirely when None.
format	"pdf"	Output format. Must be lowercase (e.g. "pdf", not "PDF") — uppercase 400s with “unknown report format.”
snapshot_filter	None	One or a list of REPORT_ALL_SNAPSHOTS_FILTER ("is_all_included") and/or REPORT_LIVE_INCLUDED_FILTER ("is_live_included"), sent as parameters.test_snapshot_filter.
excluded_sections	None	Section names (a template's own details.sections[].name) to omit — typically the output of find_unsupported_report_sections().
output_path	None	Local path to download the finished report to. Without it, the raw report object (including its id) is returned instead.
**extra	—	Additional top-level report body fields (parameters, page_layout, metadata, ...) merged into the request body as-is; if extra["parameters"] is also given, snapshot_filter=/excluded_sections= are merged into it rather than replacing it.

```
python - the minimal call
```

```
report = iq.generate_report(
    template["id"], title="Nightly regression",
    database_id=test["id"], format="pdf")
print(report["id"], report.get("status"))
```

METHOD	SIGNATURE	DESCRIPTION
get_report	get_report(report_id)	Fetch a generated report's record, including its status field.
download_report	download_report(report_id, save_as=None)	Download a finished report's file — to disk if save_as is given, else returns the raw bytes.

8.4 Downloading a Completed Report

Report generation on `orion-res` is asynchronous. For anything beyond a trivially small report, wait for it to finish before downloading: omit `output_path=` from `generate_report()`, poll `get_report(id)`'s `status` field until it leaves `queued` / `generating`, and only then call `download_report()`:

```
examples/generate_report.py - poll-then-download pattern
```

```
report = iq.generate_report(
    template["id"], title="tcicrestclient example -- wait then download",
    database_name=test["name"], excluded_sections=unsupported)

final_report, timed_out = wait_for_report(iq, report["id"])
if not timed_out:
    iq.download_report(final_report["id"], save_as="report_waited.pdf")
```

tcicrestclient Python API Sample Usages

9.1 Introduction

`tcicrestclient/examples/` ships 25 runnable, self-contained scripts. Every one guards a destructive operation behind an explicit typed confirmation, falls back to `.env` /environment configuration, and can be run directly once `tcicrestclient` is installed:

```
python examples/<script>.py
```

This chapter groups them by area and highlights each one's core call pattern.

9.2 Connecting and Discovering

SCRIPT	DEMONSTRATES
<code>discover_local.py</code>	The four ways to address a standalone server: <code>base_url=</code> , <code>host=/port=</code> , <code>install_dir=</code> , and a <code>timeout=</code> override.
<code>discover_aion.py</code>	AION login/discovery, <code>node/port</code> overrides, and a deliberately wrong password raising <code>IQConnectionError</code> .
<code>list_databases_labserver.py</code>	Connecting directly to a lab server by fixed <code>host:port</code> , with <code>use_https=/verify=</code> for a self-signed certificate.
<code>list_databases_aion.py</code>	Listing every database on an AION-hosted deployment, with a note that an empty result usually means discovery landed on the wrong AION instance.

```
examples/discover_local.py
```

```
iq = IQClient(base_url="http://127.0.0.1:9200", timeout=15)
tests = iq.list_tests()
for t in tests:
    metadata = t.get("metadata", {})
    print("    %s %-14s owner=%-35s" % (
        t["id"], t["name"], metadata.get("test.owner", "-")))
```

9.3 Managing Test Databases

SCRIPT	DEMONSTRATES
<code>list_tests_by_owner.py</code>	<code>list_tests(owner=...)</code> plus <code>get_test()</code> for full metadata.
<code>manage_test_database.py</code>	Destructive <code>rename_test()/delete_test()</code> gated behind a typed confirmation phrase.
<code>rename_test_roundtrip.py</code>	A safe rename-then-revert round trip, reverting in a <code>finally</code> block.
<code>bulk_cleanup_databases.py</code>	Preview-then-confirm bulk cleanup by size and by age (5.5).
<code>inspect_database_schema.py</code>	Full schema walk: tables, fields (all and per-table), one table's schema, and summary stats.

9.4 Views and Queries

SCRIPT	DEMONSTRATES
<code>run_view_query.py</code>	The core <code>query()</code> workflow — view/column discovery, filters, sort, limit, <code>raw_result</code> , <code>snapshot_name</code> , and a <code>definition=</code> fallback.
<code>run_live_query.py</code>	Polling a live table (7.3); shows <code>snapshot_name=</code> rejected against live data.
<code>run_boxplot_query.py</code> / <code>run_histogram_query.py</code>	The dict-of-rows return shape for “multi” view types.
<code>run_pie_chart_query.py</code> / <code>run_xy_chart_query.py</code>	Chart view types that return a plain row list; differing <code>snapshot_name=</code> behavior between the two.
<code>run_json_definition_query.py</code>	Running a raw query supplied as a JSON string via <code>definition=</code> , with modifiers layered on top.
<code>query_modifiers.py</code>	All four generic modifiers — <code>filters</code> , <code>sort</code> , <code>group_by</code> , <code>time_range</code> — against one view.
<code>multi_database_query.py</code>	Per-call <code>database_id=</code> overrides across several tests without disturbing the client’s default.
<code>fetch_view_data.py</code>	Dynamic column/type discovery, then exercising every generic <code>query()</code> feature against columns that actually support it.
<code>manage_views.py</code>	Full view CRUD: clone, save, query, delete (6.3).
<code>modify_query_definition.py</code>	Editing a raw definition dict directly — an OR-combined filter, a removed projection, an overwritten order list — for cases the structured modifiers can’t express, plus manual “unknown attribute” repair.
<code>auto_repair_demo.py</code>	<code>auto_repair</code> and <code>last_dropped_columns</code> (7.6).
<code>aion_end_to_end.py</code>	A full walkthrough on an AION-hosted deployment, showing identical client behavior once connected.

9.5 Reports

SCRIPT	DEMONSTRATES
<code>generate_report.py</code>	Excluding unsupported sections, <code>snapshot_filter</code> options, and the poll-then-download pattern (8.4).
<code>generate_report_labserver.py</code>	The same excluded-sections and poll-then-download flow, generating a report for a specific <code>database_id</code> on a named lab server reached over HTTPS with a self-signed certificate (2.4's <code>verify=False</code>), plus a <code>page_layout=</code> override and a stricter completion check that only downloads once the report's status reaches a success state (e.g. "done"/"complete").

Testing Automation Built on tcicrestclient

10.1 Introduction

`tcicrestclient`'s own automated test suite mocks 100% of HTTP traffic — no running `orion-res` server is required to run it, and the same pattern is recommended for automation you build on top of `tcicrestclient`.

10.2 Prerequisites

```
from tciclient/  
  
pip install -e ".[test]"  
pytest
```

The `test` extra installs `pytest>=7.0` and `responses>=0.23` alongside the package itself.

10.3 Mocking orion-res with responses

Every HTTP call in the suite is intercepted with the `responses` library. Two equivalent patterns appear throughout the codebase: a shared `mocked_responses` pytest fixture wrapping `responses.RequestsMock`, or the `@responses.activate` decorator directly on a test function.

```
tests/conftest.py – the shared fixture pattern  
  
def test_list_tests(mocked_responses, iq_client):  
    mocked_responses.add(  
        responses_lib.GET, BASE + "/databases",  
        json=[{"id": "1", "name": "demo"}], status=200)  
  
    tests = iq_client.list_tests()  
    assert tests[0]["id"] == "1"
```

10.4 Fixtures and Environment Isolation

`conftest.py` provides an `iq_client` fixture — an `IQClient(base_url="http://fake-orion-res:9200", load_env=False)` with every `TCIQ_*` environment variable explicitly deleted via `monkeypatch`, so a real `.env` file or an ambient variable on the test-runner machine can never leak into a test. `tests/fixtures.py` holds real, trimmed captured request/response shapes; larger real captures live under `tests/data/*.json` and are loaded through helpers that call `pytest.skip()` gracefully when a given capture file isn't present, rather than failing the whole suite.

Common edge cases worth reproducing in your own tests, all confirmed and covered by `tcicrestclient`'s own suite: an empty `list_views()` / `list_table_names()` result returning `[]` rather than raising; a missing `"summary"` key returning `{}`; a 401 during AION login raising `IQConnectionError`; and a direct (non-AION) connection sending *no* `Authorization` header at all.

IQClient Method Reference

Every public method on `IQClient`, grouped by area. See the chapter listed for full parameter descriptions.

Connection

METHOD	DESCRIPTION	CHAPTER
<code>base_url</code>	Property: the resolved orion-res base URL.	3

Tests & Databases

METHOD	DESCRIPTION	CHAPTER
<code>list_tests(owner=, detail=)</code>	List tests, optionally filtered by owner (client-side).	5
<code>get_test(database_id=)</code>	Full metadata for one test.	5
<code>use_test(database_id)</code>	Set the default database id.	5
<code>rename_test(new_name, database_id=)</code>	Rename a database.	5
<code>delete_test(database_id=)</code>	Permanently delete a database.	5
<code>get_database_schema(database_id=)</code>	Full schema record.	5
<code>list_table_names(database_id=) /</code> <code>get_database_tables(...)</code>	List tables (result & dimension sets).	5
<code>list_fields(table_name=, database_id=)</code>	List fields, all or per-table.	5
<code>get_table_schema(table_name, database_id=)</code>	One table's schema.	5
<code>get_database_summary(database_id=)</code>	Row count & storage size.	5
<code>list_databases_over_size(min_size_kb)</code>	Preview databases over a size threshold.	5
<code>delete_databases_over_size(min_size_kb, dry_run=)</code>	Bulk-delete by size.	5
<code>list_databases_older_than(days)</code>	Preview databases older than N days.	5
<code>delete_databases_older_than(days, dry_run=)</code>	Bulk-delete by age.	5

Views

METHOD	DESCRIPTION	CHAPTER
<code>list_views(timeout=)</code>	List all views.	6
<code>get_view(view_id, timeout=)</code>	Get one view by id.	6
<code>find_view(name, timeout=)</code>	Find a view by display name.	6
<code>list_view_columns(name, ...)</code>	List a view's available/active columns.	6
<code>save_view(name, details=, description=, definition=)</code>	Create or update a view.	6
<code>delete_view(view_id=, name=, timeout=)</code>	Delete a view.	6

Queries

METHOD	DESCRIPTION	CHAPTER
<code>query(name=, definition=, ...)</code>	Run a table, live, snapshot, or chart query. The only query method.	7
<code>last_dropped_columns</code>	Attribute: columns auto-repair dropped on the most recent call.	7

Reports

METHOD	DESCRIPTION	CHAPTER
<code>list_report_templates()</code>	List all report templates.	8
<code>get_report_template(template_id)</code>	Get one report template.	8
<code>find_unsupported_report_sections(template_id, timeout=)</code>	Find sections referencing an unsafe view type.	8
<code>generate_report(template_id, ...)</code>	Generate (and optionally download) a report.	8
<code>get_report(report_id)</code>	Get a generated report's record/status.	8
<code>download_report(report_id, save_as=)</code>	Download a finished report's file.	8

Module-level Helpers & Constants

SYMBOL	DESCRIPTION	CHAPTER
<code>merge_modifiers(definition, filters=, sort=, group_by=, time_range=, limit=, resolve_field=)</code>	Layer filters/sort/group_by/time_range/limit onto any query definition.	7
<code>rows_to_dicts(result)</code>	Zip a raw result's columns and rows into row dicts.	7
<code>DEFAULT_QUERY_LIMIT</code>	1000 — default row cap for <code>query()</code> .	3
<code>IQError</code> and subclasses	The complete exception family.	3
<code>__version__</code>	Installed <code>tcicrestclient</code> package version.	2

References

Documents distributed alongside `tciqurestclient`, in the order a new reader is expected to need them:

`tciqurestclient/QUICKSTART.md`

The fastest path — five steps, no other reading required.

`tciqurestclient/README.md`

Short overview, install instructions, and the discovery precedence table.

`tciqurestclient/QUICKGUIDE.md`

Full task-by-task walkthrough for a checked-out repository, linking directly into `examples/`.

`tciqurestclient/GETTING_STARTED.md`

A self-contained equivalent of the Quick Guide for a reader with only `pip install` and no repository clone, including a PyPI-publishing checklist for maintainers.

`tciqurestclient/API_REFERENCE.md`

The complete symbol-by-symbol reference — every parameter, return value, and exception, organized by symbol rather than by task.

`tciqurestclient/TESTING.md`

Automated and manual/integration testing guide, including the pre-release checklist.

`tciqurestclient/CHANGELOG.md`

Per-version release notes.

HANDOVER.md

Full architecture writeup, requirement-status table, and the running, date-stamped log of confirmed-vs-assumed findings this manual's status badges are drawn from.

tciclient/examples/

24 runnable example scripts, one per usage pattern covered in Chapter 9.

External references:

Requests

docs.python-requests.org — the HTTP library `tcicrestclient` is built on.

python-dotenv

Loads `.env` file entries as environment variables for configuration.

PyYAML

Parses `orion-res.yaml` during local install-directory discovery.

pytest

/

responses

Test runner and HTTP-mocking library used by `tcicrestclient`'s own test suite and recommended for automation built on top of it.

